Beyond the QA Bottleneck

Building confidence in React Native with AI



John Adib • MrAdib.com

Engineering Manager at Zapp

React Native London Meetup • Thursday 22 October 2026
Funding Circle • 71 Queen Victoria Street, London

Regressions.
Every release.

We kept returning to behaviour we had already checked.

Tests existed.
Nobody ran them.

A small test suite. End-to-end checks outside CI.

More than three days of QA.

Engineering, product and UX/design.
Many people paused their own work.

Every release waited days for approval.

A chain of sign-offs held up the whole release.

Projects postponed by over a quarter.

All the existing projects had slipped by more than a quarter.



John Adib

The story of an EM in the era of AI

2× co-founder. Former CTO. Engineering leader and mentor.

18 years building and improving systems. I love moving mountains.

MrAdib.com

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verification

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 Sentry

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

Eight skills. One development loop.

Each skill has a responsibility and an output.

cook
Coordinates the task

dig
Researches the behaviour

michelin
Implements and verifies

parole
Evaluates review feedback

sherlock
Discovers the repository

mastermind
Plans for approval

mugshot
Captures device evidence

saul
Communicates the result

My engineering standards, written down

Michelin makes those standards part of every change.

Reuse and consistency	Build on existing code, patterns and tools.
Separation of concerns	Keep UI, state and business logic focused.
Quality bar	Meaningful tests. Readable code. Verified results.

01 Skills

Make good engineering habits repeatable.

02 **Testing**

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verifications

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 Sweep

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

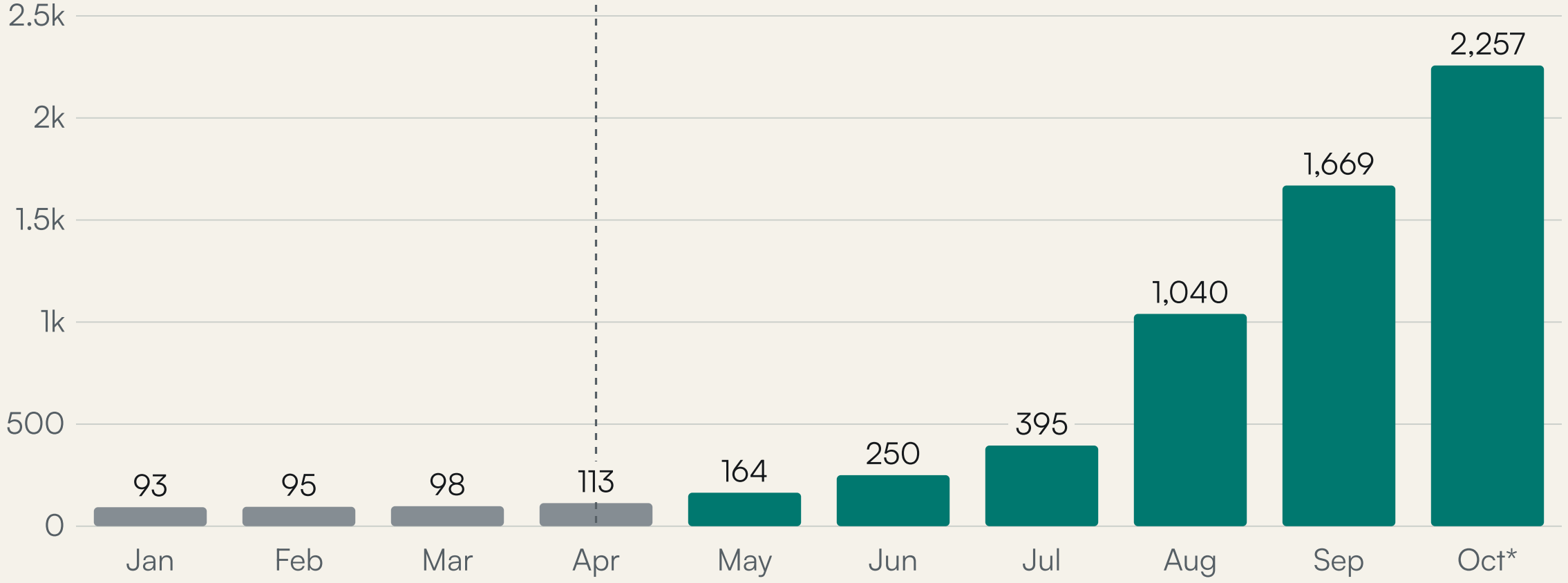
The testing bar, written into the codebase

Protect what works. Raise the bar with every change.

New code needs tests	No test, no acceptance.
Bug fixes prove both cases	The test fails before the fix and passes after it.
Titles explain behaviour	Name the expected result in plain language.
The bar keeps rising	Keep existing checks passing. Add protection as we go.

113 to 2,257 unit-test files

Joined in April 2026



Month-end snapshots through September, then 8 October 2026. File count is not coverage.

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verifications

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 Sentry

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

Every PR goes through CI

Failed checks block the pipeline until they pass.

Run the checks	Lint, types, tests and build on every PR.
Parole evaluates	Watch CI results. Investigate each failure.
Michelin fixes	Make the correction. Commit, push and run again.

The loop repeats until the required checks pass.

The quality bar keeps rising

99.26%

Line coverage, 8 October 2026

Measured scope changed. More tested lines, fewer eligible lines.
CI now preserves this baseline.

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verification

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 Sentry

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

Maestro checks the app while we sleep

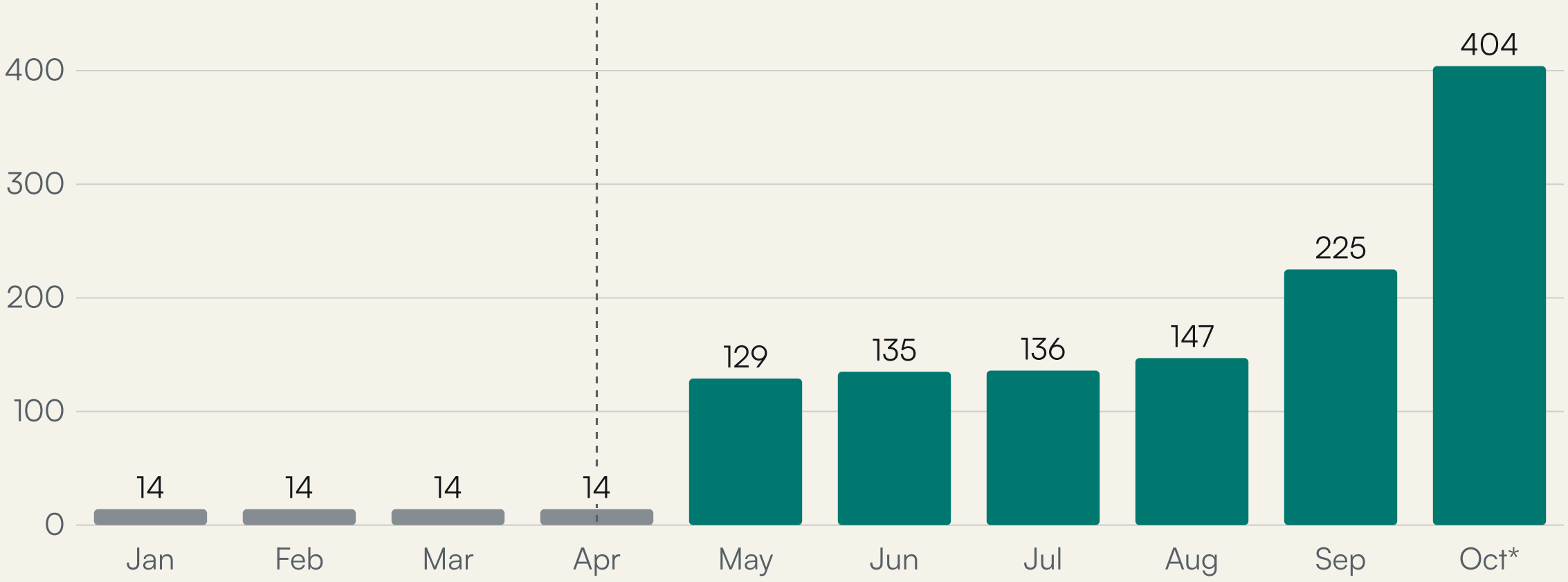
Android and iOS, with evidence of every failure.

Selected PRs	Run smoke tests when the relevant label is applied.
Every night	33 configured suite jobs across iOS and Android.
Investigate failures	Watch the video, inspect logs and fix the affected flow.

Attach recordings and logs so people and AI can investigate.

14 to 404 Maestro flow files

Joined in April 2026



Month-end snapshots through September, then 8 October. Includes shared flows and suites.

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verification

Maestro checks on iOS and Android. Recorded results and failures.

05 **AI reviewers**

Three reviewers on every PR. Shared mobile guidelines.

06 Sweep

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

Three AI reviewers. One mobile specialist.

Review feedback arrives without waiting days for a person.

CodeRabbit, Qodo, cubic	Three review perspectives on every PR.
Custom Cursor reviewer	An hourly review focused on React Native.
A shared quality bar	All reviewers use the guidelines in the repository.

Review guidelines live with the code

Every AI reviewer starts with the same mobile context.

React Native standards	Mobile best practices and platform behaviour.
Engineering expectations	Reuse existing code. Separate concerns. Require tests.
Guidelines evolve	Clarify missed context and improve future reviews.

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verification

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 **Sentry**

Turn production issues into the next regression check.

07 Delivery

Ship small changes. Activate features separately.

Turn an issue into a regression check

01 Investigate a recorded issue

02 Reproduce the relevant behaviour

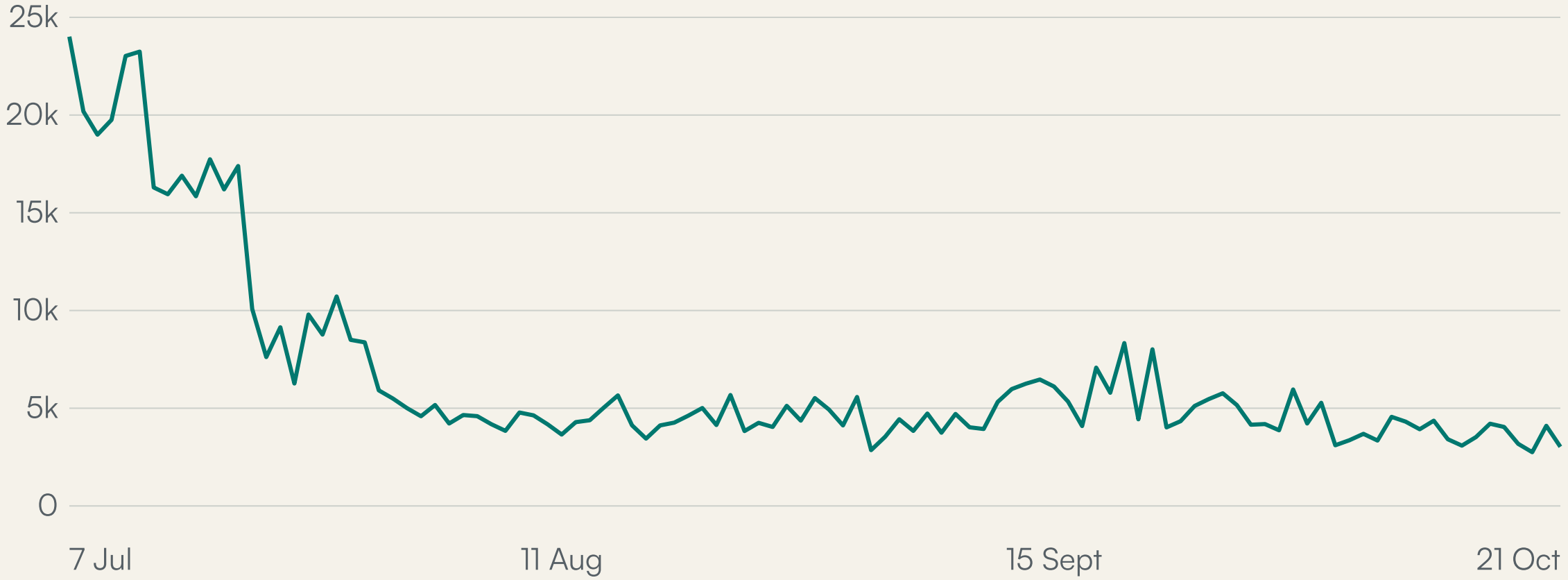
03 Make a bounded correction

04 Add a regression check

05 Return evidence to review

A resurfacing issue becomes a focused development task.

Fewer recorded errors, with variation



65% fewer events: 7 Jul to 5 Aug vs 22 Sep to 21 Oct 2026. Both windows are 30 complete days.
All environments. Fixes, filters and sampling affect counts. No usage denominator.

01 Skills

Make good engineering habits repeatable.

02 Testing

Make each check protect real behaviour.

03 CI

Check every PR. Block the pipeline on failure.

04 Device verification

Maestro checks on iOS and Android. Recorded results and failures.

05 AI reviewers

Three reviewers on every PR. Shared mobile guidelines.

06 Sentry

Turn production issues into the next regression check.

07 **Delivery**

Ship small changes. Activate features separately.

Canary builds and nightly betas

TestFlight for iOS. Test builds for Android.

After each merge	A canary build to try the change.
Every night	An automated beta build for testing.
Side by side	Install canary beside the normal app. Compare before and after.
Clear test instructions	Generated TestFlight notes explain what changed and what to test.

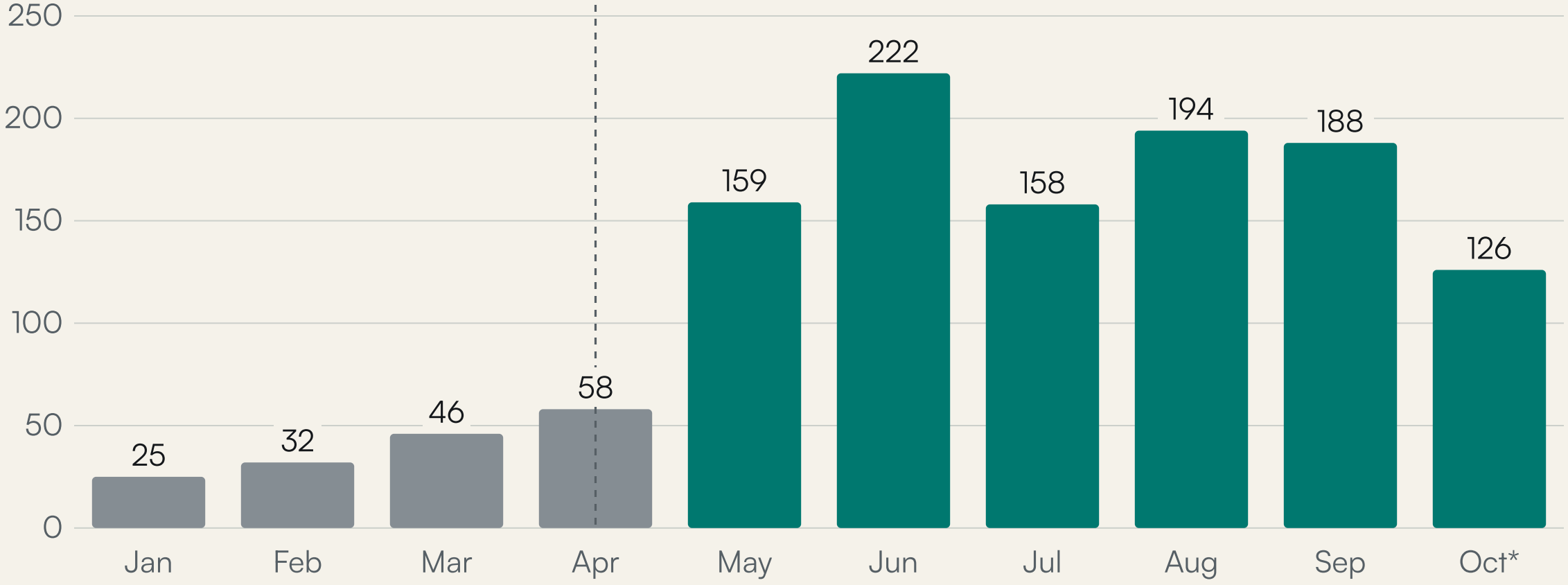
Release the code. Approve the flag.

Small changes ship behind feature flags.

Small increments	Keep unfinished features hidden behind a flag.
Seamless releases	Release verified code without the feature sign-off queue.
Focused approval	Approve activating the feature when it is ready.

The development rhythm changed

Joined in April 2026



App and design-system PRs. October* covers 1 to 8 October only. Activity, not productivity.

There’s another talk in what we left out

A few months of change across the team, codebase and business.

Design system	A shared UI language and a consistent quality bar.
Monorepo	Shared code and coordinated changes.
Ways of working	Clearer ownership and everyday engineering habits.
Product engineering	Broader ownership, from product decisions to delivery.
AI development	Adoption across planning, code, review and verification.
An AI harness	Context, skills, tools and checks around the agents.

What I’d take into another team

Find the bottleneck	Start where repeated work slows delivery.
Write down the bar	Make quality expectations clear to people and AI.
Close the feedback loop	Connect checks, evidence and fixes.
Adapt the rollout	Choose tools and release rules that fit your team.

Keep the principles. Adapt the implementation.

Confidence in AI-assisted development

Seven practices for mobile development. One continuous loop.

Skills	Repeat good engineering habits.
Testing	Protect behaviour with regression checks.
CI	Check every PR. Block failures until fixed.
Device verification	Maestro smoke and nightly checks. Video evidence.
AI reviewers	Mobile guidelines. Evaluate feedback. Fix valid issues.
Sentry	Turn production issues into regression checks.
Delivery	Canary on merge. Nightly beta. Feature flags.

MrAdib.com/@

Less repetitive QA. More confidence. Faster delivery.

John Adib • MrAdib.com